

Etapa 2: Panificación de estrategias de sharding y replica sets

Caso de estudio: Ecommify

Autores: Daniel Porras, Oscar Clavijo, Camilo Porras

Fecha: junio 2026

1. Introducción

El presente entregable desarrolla la segunda actividad de la Unidad 3, orientada a la planificación de estrategias de escalabilidad horizontal en MongoDB mediante sharding y replica sets para la plataforma Ecommify.

Ecommify es una plataforma de comercio electrónico híbrida diseñada bajo una arquitectura de microservicios y arquitectura orientada a eventos. En el documento técnico de diseño se establece que PostgreSQL se utiliza para procesos críticos del negocio, como usuarios, órdenes, pagos e inventario, debido a sus garantías transaccionales ACID. Por otro lado, MongoDB se emplea para componentes con mayor flexibilidad estructural y alto volumen de datos, como catálogo extendido, comportamiento de usuario, recomendaciones, reseñas y logs de búsqueda.

El objetivo principal es proponer una arquitectura distribuida para MongoDB que permita soportar crecimiento horizontal, alta disponibilidad, tolerancia a fallos y buen rendimiento para las operaciones no transaccionales de Ecommify.

2. Estudio de arquitectura distribuida de MongoDB

2.1 Replica Sets

Un replica set en MongoDB es un conjunto de nodos que mantienen copias de los mismos datos. Su propósito principal es garantizar alta disponibilidad, tolerancia a fallos y continuidad del servicio ante la caída de un nodo. La arquitectura básica de un replica set incluye los siguientes roles:

Rol	Descripción
Primary	Nodo principal que recibe todas las escrituras
Secondary	Nodo que replica los datos desde el Primary
Arbiter	Nodo que participa en elecciones, pero no almacena datos

2.2 Rol Primary

El nodo Primary es el nodo principal del replica set. Todas las operaciones de escritura se realizan inicialmente sobre este nodo. Posteriormente, los cambios se replican hacia los nodos Secondary. En el contexto de Ecommify, las operaciones como creación de productos, actualización de precio, actualización de métricas del catálogo o registro de reseñas deberían pasar por el nodo Primary cuando se requiere mayor confiabilidad.

2.3 Rol Secondary

Los nodos Secondary mantienen una copia de los datos del Primary. Estos nodos pueden atender lecturas dependiendo de la configuración de read preference. En Ecommify, los nodos Secondary pueden ser útiles para descargar consultas de lectura del catálogo, reportes, recomendaciones o consultas analíticas que no requieren consistencia inmediata.

2.4 Rol Arbiter

El Arbiter participa en procesos de elección cuando el Primary falla, pero no almacena datos. Aunque puede ayudar a mantener un número impar de votos, no aporta redundancia de datos.

Para Ecommify no se recomienda utilizar Arbiter en la arquitectura objetivo, ya que se trata de una plataforma que requiere disponibilidad y resiliencia. Es preferible contar con tres nodos con datos completos en diferentes zonas de disponibilidad.

3. Read Preferences

Las read preferences permiten definir desde qué nodo se ejecutan las operaciones de lectura. Esta decisión afecta directamente el rendimiento, la disponibilidad y el nivel de consistencia de las consultas.

Read Preference	Descripción	Uso recomendado
primary	Lee siempre desde el nodo Primary	Operaciones críticas o sensibles a datos recientes
primaryPreferred	Lee desde Primary, pero puede leer desde Secondary si el Primary no está disponible	Consultas que prefieren consistencia, pero toleran fallback

secondary	Lee solo desde nodos Secondary	Consultas analíticas o no críticas
secondaryPreferred	Lee desde Secondary, pero usa Primary si no hay Secondary disponible	Catálogo, reseñas y consultas de alto volumen
nearest	Lee desde el nodo con menor latencia	Escenarios geográficamente distribuidos

3.1 Estrategia propuesta de read preferences para Ecommify

Operación	Colección	Read Preference	Justificación
Consulta general del catálogo	product_catalog	secondaryPreferred	Reduce carga sobre el Primary y mejora rendimiento de lectura
Detalle de producto	product_catalog	secondaryPreferred	Tolera una pequeña ventana de inconsistencia
Consulta por categoría	product_catalog	secondaryPreferred	Es una consulta frecuente del catálogo
Consulta de reseñas	product_reviews	secondaryPreferred	Las reseñas no son críticas transaccionalmente
Recomendaciones	recommendations	secondary	Se generan de forma analítica y toleran desfase
Historial de navegación	user_behavior	secondary	Uso analítico de alto volumen
Validación de precio recién actualizado	product_catalog	primary	El precio puede impactar decisiones de compra
Actualización de producto	product_catalog	primary	Requiere leer y escribir información reciente

Las operaciones de pagos, órdenes e inventario no se asignan a MongoDB en esta arquitectura, ya que el documento técnico de Ecommify las ubica en PostgreSQL por requerir consistencia fuerte.

4. Write Concerns

Los write concerns determinan el nivel de confirmación que MongoDB debe recibir antes de considerar exitosa una escritura. Esta decisión afecta la durabilidad, la latencia y el nivel de seguridad de los datos.

Write Concern	Descripción	Garantía
w:1	Confirma escritura solo en el Primary	Baja latencia, menor garantía
majority	Confirma escritura en la mayoría de nodos	Mayor durabilidad
w:2	Confirma escritura en dos nodos	Garantía intermedia
journaled	Confirma escritura en journal	Mayor protección ante fallos

4.1 Estrategia propuesta de write concerns para Ecommify

Operación	Colección	Write Concern	Justificación
Crear producto	product_catalog	majority	El producto debe quedar persistido de forma confiable
Actualizar precio	product_catalog	majority	El precio es sensible para el negocio
Actualizar atributos de producto	product_catalog	majority	Evita pérdida de cambios en el catálogo
Registrar reseña	product_reviews	majority	Información visible para otros usuarios
Registrar evento de navegación	user_behavior	w:1	Evento masivo, no crítico individualmente
Registrar búsqueda	search_logs	w:1	Dato analítico que puede tolerar pérdida parcial
Actualizar recomendación	recommendations	w:1	Puede recalcularse posteriormente
Actualizar métricas aproximadas	product_catalog	w:1	Métricas como vistas o popularidad pueden reprocesarse

La estrategia diferencia operaciones visibles o sensibles para el negocio frente a eventos analíticos masivos. Para información crítica del catálogo se propone `majority`; para eventos de alto volumen y bajo impacto individual se permite `w:1`.

5. Conceptos de Sharding

Sharding es el mecanismo de MongoDB para distribuir datos horizontalmente entre múltiples shards. Cada shard almacena una porción de los datos, lo que permite escalar cuando una sola instancia o replica set no es suficiente para soportar el volumen de datos o la carga de consultas.

5.1 Componentes de una arquitectura con sharding

Componente	Función
Shards	Almacenan subconjuntos de datos
Config Servers	Almacenan metadatos del cluster
Mongos Routers	Reciben las consultas de la aplicación y las enrutan al shard correspondiente

5.2 Shards

Los shards son los nodos o replica sets donde se almacenan los datos distribuidos. Cada shard contiene una parte del total de documentos de una colección. En Ecommify, los shards serían especialmente útiles para colecciones de alto crecimiento como `user_behavior`, `search_logs`, `recommendations`, `product_reviews` y, en menor medida, `product_catalog`.

5.3 Config Servers

Los Config Servers almacenan los metadatos del cluster, incluyendo la distribución de chunks, la ubicación de los datos y la configuración del sharding. Son componentes críticos, ya que permiten que los routers conozcan dónde se encuentra cada porción de datos.

5.4 Mongos Routers

Los Mongos Routers actúan como punto de entrada para las aplicaciones. La aplicación no necesita saber en qué shard está cada documento; envía la consulta al router y este la redirige al shard correspondiente. En Ecommify, los microservicios que consultan MongoDB se conectarían a través de los Mongos Routers.

6. Tipos de Shard Keys

La shard key es el campo o conjunto de campos que MongoDB utiliza para distribuir los documentos entre shards. Elegir una shard key adecuada es una de las decisiones más importantes en el diseño distribuido.

- **Ranged Shard Key:** Una shard key por rango distribuye los documentos según rangos de valores.

Aplicación en Ecommify: Puede servir para search_logs o eventos históricos, pero debe combinarse con otra estrategia para evitar concentración de escrituras recientes.

- **Hashed Shard Key:** Una shard key hashed distribuye los documentos usando un hash del campo seleccionado.

Aplicación en Ecommify: Es útil para user_behavior, ya que los eventos de usuarios pueden crecer de forma masiva y se necesita distribución uniforme.

- **Compound Shard Key:** Una shard key compuesta utiliza varios campos para distribuir y consultar los datos.

Aplicación en Ecommify: Es la alternativa más apropiada para product_catalog, porque las consultas suelen filtrar por categoría, región del vendedor, precio, calificación o popularidad.

7. Análisis de distribución de datos para Ecommify

Durante la implementación de la Etapa 1 se insertaron 1000 productos en la colección product_catalog. Posteriormente, se ejecutaron aggregation pipelines para analizar la distribución de datos por categoría y por región del vendedor.

7.1 Distribución de productos por categoría

El pipeline ejecutado para analizar la distribución fue:

```
7.1 Distribución de productos por categoría

[24] pipeline = [
  {
    "$group": {
      "_id": "$category",
      "total_products": {"$sum": 1}
    }
  },
  {
    "$sort": {"total_products": -1}
  }
]

for result in product_catalog.aggregate(pipeline):
    print(result)

...
{'_id': 'Deportes', 'total_products': 228}
{'_id': 'Libros', 'total_products': 214}
{'_id': 'Tecnología', 'total_products': 218}
{'_id': 'Hogar', 'total_products': 178}
{'_id': 'Ropa', 'total_products': 178}
```

7.2 Análisis de concentración por categoría

Para identificar posibles hotspots se calcula un índice de concentración usando la suma de cuadrados de las participaciones de cada categoría.

El valor mínimo esperado para cinco categorías perfectamente balanceadas sería cercano a 0.20. El resultado obtenido, 0.2024, indica una distribución bastante uniforme para los datos de prueba. Por lo tanto, en el dataset generado no se observa una concentración crítica por categoría.

Sin embargo, en un escenario productivo real, categorías como Tecnología o Ropa podrían crecer más rápido y concentrar una mayor cantidad de consultas. Por esta razón, no se recomienda usar únicamente category como shard key.

7.3 Distribución de productos por región del vendedor

El pipeline ejecutado para analizar la distribución regional fue:

```
[25] ✓ Os # 7.3 Distribución de productos por región del vendedor

pipeline = [
  {
    "$group": {
      "_id": "$seller_region",
      "total_products": {"$sum": 1}
    }
  },
  {
    "$sort": {"total_products": -1}
  }
]

for result in product_catalog.aggregate(pipeline):
    print(result)

... {'_id': 'CO-BOG', 'total_products': 239}
{'_id': 'CO-ANT', 'total_products': 207}
{'_id': 'CO-VAC', 'total_products': 201}
{'_id': 'CO-ATL', 'total_products': 187}
{'_id': 'CO-SAN', 'total_products': 166}
```

7.4 Análisis de concentración regional

La distribución regional también es equilibrada en los datos de prueba. No se observa una concentración excesiva en una sola región. Sin embargo, en un ambiente productivo podrían

existir regiones con mayor volumen comercial, como Bogotá o Antioquia, por lo que se recomienda considerar seller_region dentro de una shard key compuesta.

8. Evaluación de posibles shard keys para product_catalog

Opción 1: Shard key por categoría

No se recomienda como única shard key porque puede concentrar carga en categorías populares.

Opción 2: Shard key por región del vendedor

No se recomienda como única shard key, pero sí como parte de una shard key compuesta.

Opción 3: Shard key por _id hashed

Es útil para distribución, pero no aprovecha los patrones de consulta del catálogo.

Opción 4: Compound shard key por categoría, región e identificador hash

Esta es la opción recomendada para product_catalog, porque balancea eficiencia de consultas y distribución uniforme.

9. Shard key recomendada para product_catalog

Se recomienda utilizar una shard key compuesta ya que la colección product_catalog se consulta principalmente por categoría, atributos del producto, precio, calificación, ventas y región del vendedor. En este contexto, una clave compuesta permite combinar criterios funcionales y técnicos.

El campo category apoya búsquedas frecuentes del catálogo. El campo seller_region permite distribuir los productos según ubicación del vendedor y puede ayudar a consultas logísticas o regionales. El campo _id con hash mejora la distribución dentro de cada combinación de categoría y región.

Esta combinación reduce el riesgo de concentrar todos los productos de una categoría popular en un único shard.

9.1 Índices complementarios recomendados

Además de la shard key, se recomienda mantener índices secundarios para consultas frecuentes:

```
[27]
✓ Os # Índices complementarios en PyMongo

db.product_catalog.create_index([("category", 1), ("price", 1)])
db.product_catalog.create_index([("tags", 1)])
db.product_catalog.create_index([("seller_id", 1)])
db.product_catalog.create_index([("seller_region", 1)])
db.product_catalog.create_index([("ratings.average", -1)])
db.product_catalog.create_index([("metrics.total_units_sold", -1)])
db.product_catalog.create_index([("attributes.ram", 1)])

print("Índices creados correctamente")

... Índices creados correctamente
```

Estos índices permiten soportar consultas por precio, etiquetas, vendedor, calificación, ventas y atributos dinámicos.

10. Estrategia de sharding para otras colecciones de MongoDB

Aunque la colección principal de la actividad es `product_catalog`, el diseño técnico de Ecommify también contempla colecciones como `user_behavior`, `product_reviews`, `recommendations` y `search_logs`.

Colección `user_behavior`: Esta colección almacena eventos de navegación de usuarios y puede crecer de forma extremadamente alta. Se recomienda aplicar Bucket Pattern y una shard key basada en usuario y periodo.

Shard key propuesta:

```
{
  "userId": "hashed",
  "period": 1
}
```

Colección `product_reviews`: Esta colección almacena reseñas completas de productos.

Shard key propuesta:

```
{
  "product_id": "hashed"
}
```

Colección search_logs: Esta colección almacena búsquedas realizadas por los usuarios.

Shard key propuesta:

```
{
  "created_at": 1,
  "_id": "hashed"
}
```

Colección recommendations: Esta colección almacena recomendaciones generadas para usuarios.

Shard key propuesta:

```
{
  "user_id": "hashed"
}
```

11. Configuración teórica de replica set

Para Ecommify se propone un replica set de tres nodos distribuidos en múltiples zonas de disponibilidad.

Nodo	Rol inicial	Zona	Descripción
mongo-rs-01	Primary	AZ-1	Recibe escrituras y operaciones críticas
mongo-rs-02	Secondary	AZ-2	Replica datos y atiende lecturas no críticas
mongo-rs-03	Secondary	AZ-3	Replica datos y permite failover

Esta configuración permite alta disponibilidad y tolerancia a fallos. Si el nodo Primary falla, los nodos Secondary pueden elegir automáticamente un nuevo Primary. Además, al estar distribuidos en varias zonas de disponibilidad, se reduce el riesgo de interrupción por caída de

una zona. No se propone Arbiter porque Ecommify requiere redundancia real de datos. Un Arbiter puede votar, pero no almacena información, por lo tanto no aporta capacidad de recuperación de datos.

12. Configuración teórica de sharded cluster

En un ambiente productivo, la arquitectura objetivo podría componerse de varios shards, donde cada shard es a su vez un replica set.

Propuesta de alto nivel:

Componente	Configuración propuesta
Shard 1	Replica set de 3 nodos
Shard 2	Replica set de 3 nodos
Shard 3	Replica set de 3 nodos
Config Servers	Replica set de 3 nodos
Mongos Routers	Mínimo 2 instancias
Zonas	Multi-AZ

Cada shard almacena una parte de los datos. Los Config Servers almacenan metadatos del cluster y los Mongos Routers reciben las solicitudes desde los microservicios de Ecommify.

Los servicios de catálogo, reseñas, recomendaciones y analítica se conectarían a través de los Mongos Routers, sin conocer directamente en qué shard reside cada documento.

13. Consistencia eventual

La consistencia eventual ocurre cuando una escritura realizada en el Primary tarda un tiempo en replicarse hacia los nodos Secondary. Durante esa ventana, una lectura desde un Secondary podría retornar datos antiguos.

A este retraso se le conoce como replication lag.

Ventana típica de inconsistencia: En condiciones normales, el replication lag suele ser bajo, pero puede aumentar por:

- Alto volumen de escrituras.
- Saturación de CPU.
- Problemas de red.
- Lentitud en disco.
- Índices costosos.
- Operaciones masivas.
- Carga analítica sobre nodos secundarios.

En Ecommify, la consistencia eventual es aceptable para recomendaciones, logs de búsqueda, navegación y métricas aproximadas. Sin embargo, debe controlarse en operaciones que impactan la experiencia del usuario.

Operaciones críticas fuera de MongoDB:

Pagos, órdenes e inventario no deben depender de consistencia eventual en MongoDB. Según el diseño técnico de Ecommify, estas operaciones pertenecen a PostgreSQL porque requieren consistencia fuerte, integridad transaccional y control ACID.

14. Mitigación de consistencia eventual

Para reducir los riesgos asociados a consistencia eventual se proponen las siguientes estrategias:

- **Lecturas desde Primary:** Las operaciones sensibles a datos recientes deben usar read preference primary.
- **Write Concern majority:** Las escrituras importantes deben usar majority para garantizar que fueron replicadas en la mayoría de nodos antes de confirmarse.
- **Causal Consistency Sessions:** Las sesiones con causal consistency permiten que una lectura posterior a una escritura observe los cambios realizados previamente por la misma sesión. Esto es útil cuando un usuario o administrador actualiza un producto y luego necesita verlo inmediatamente actualizado.
- **Reprocesamiento asíncrono:** Para recomendaciones, métricas y comportamiento de usuario, se permite reprocesamiento asíncrono basado en eventos. Esta estrategia es coherente con la arquitectura orientada a eventos de Ecommify.

- **Monitoreo de replication lag:** Se debe monitorear constantemente el replication lag. Si el retraso aumenta, puede afectar lecturas desde Secondary y generar datos desactualizados visibles para los usuarios.

15. Plan de monitoreo

Para operar MongoDB en Ecommify se recomienda monitorear métricas técnicas y funcionales.

Métrica	Objetivo
Replication lag	Detectar retrasos entre Primary y Secondary
Opcounters	Medir operaciones de lectura y escritura
Query execution time	Detectar consultas lentas
CPU usage	Identificar saturación de nodos
Memory usage	Evaluar presión de memoria
Disk I/O	Detectar problemas de almacenamiento
Connections	Controlar conexiones activas
Index usage	Validar si las consultas usan índices
Chunk distribution	Verificar balanceo entre shards
Hot shards	Identificar concentración de carga
Storage growth	Medir crecimiento de colecciones
Page faults	Identificar lecturas costosas desde disco
Network throughput	Evaluar tráfico entre nodos

Las métricas más importantes para esta arquitectura son:

- Replication lag.
- Query execution time.
- Chunk distribution.

- Hot shards.
- Index usage.
- Storage growth.
- CPU y memoria.

Estas métricas permiten detectar problemas de consistencia eventual, consultas lentas, mala distribución de datos o necesidad de ajustar índices.

16. Limitaciones del free tier de MongoDB Atlas

Durante la actividad se utilizó MongoDB Atlas en su modalidad gratuita M0. Este entorno es adecuado para pruebas académicas, conexión desde Google Colab, inserción de documentos, consultas y validación del modelo. Sin embargo, el free tier tiene limitaciones para implementar una arquitectura completa de producción.

Por las restricciones del free tier, la estrategia de sharding, replica sets y write concerns se documenta como diseño teórico. La validación práctica se realiza mediante el análisis de distribución de los 1000 productos insertados y los aggregation pipelines ejecutados en la Etapa 1.

En un ambiente productivo real, Ecommify debería utilizar un cluster dedicado de MongoDB Atlas con sharding habilitado, configuración multi-AZ, backups automáticos, alertas avanzadas, control de acceso robusto y monitoreo continuo.

17.Código de apoyo para análisis de concentración

Concentración por categoría:

```
[28] ✓ On # Concentración por categoría

category_distribution = list(product_catalog.aggregate([
    {
        "$group": {
            "_id": "$category",
            "total_products": {"$sum": 1}
        }
    }
]))

total = sum(item["total_products"] for item in category_distribution)

hhi_category = sum((item["total_products"] / total) ** 2 for item in category_distribution)

print("Distribución por categoría:")
for item in category_distribution:
    participation = item["total_products"] / total
    print(item["_id"], item["total_products"], round(participation * 100, 2), "%")

print("HHI categoría:", round(hhi_category, 4))

... Distribución por categoría:
Wegar 178 17.8 %
Libros 214 21.4 %
Ropa 170 17.0 %
Tecnología 210 21.0 %
Deportes 228 22.8 %
HHI categoría: 0.2025
```

Concentración por región:

```
[29] ✓ On # Concentración por región

region_distribution = list(product_catalog.aggregate([
    {
        "$group": {
            "_id": "$seller_region",
            "total_products": {"$sum": 1}
        }
    }
]))

total = sum(item["total_products"] for item in region_distribution)

hhi_region = sum((item["total_products"] / total) ** 2 for item in region_distribution)

print("Distribución por región:")
for item in region_distribution:
    participation = item["total_products"] / total
    print(item["_id"], item["total_products"], round(participation * 100, 2), "%")

print("HHI región:", round(hhi_region, 4))

... Distribución por región:
CO-ATL 187 18.7 %
CO-ANT 207 20.7 %
CO-VAC 201 20.1 %
CO-SAN 166 16.6 %
CO-BOG 239 23.9 %
HHI región: 0.2029
```

Interpretación del HHI:

HHI cercano a 0.20 con cinco grupos indica distribución equilibrada.

HHI alto indica concentración y posible hotspot.

HHI cercano a 1.0 indica concentración extrema en un solo grupo.

18. Comandos teóricos de configuración

Los siguientes comandos son teóricos y corresponden a un ambiente MongoDB con sharding habilitado. No necesariamente pueden ejecutarse en el cluster gratuito M0.

Habilitar sharding en la base de datos:

```

  Configuración teórica de sharding

  El cluster utilizado para esta actividad corresponde a MongoDB Atlas Free Tier M0, por lo cual no se ejecuta una configuración real de sharding.
  La siguiente configuración se documenta de forma teórica para un ambiente productivo con MongoDB sharded cluster.
  En un ambiente productivo, primero se habilitaría sharding sobre la base de datos:

  sh.enableSharding("ecommyfy_db")

  category_distribution = list(product_catalog.aggregate([
    {
      "$group": {
        "_id": "$category",
        "total_products": {"$sum": 1}
      }
    }
  ]))

  total = sum(item["total_products"] for item in category_distribution)
  hhi_category = sum((item["total_products"] / total) ** 2 for item in category_distribution)

  print("Distribución por categoría:")
  for item in category_distribution:
    participation = item["total_products"] / total
    print(item["_id"], item["total_products"], round(participation * 100, 2), "%")

  print("HHI categoría:", round(hhi_category, 4))

  ... Distribución por categoría:
  Tecnología 210 21.0 %
  Ropa 170 17.0 %
  Hogar 178 17.8 %
  Deportes 228 22.8 %
  Libros 214 21.4 %
  HHI categoría: 0.2025

```

Crear índice para la shard key:

```

# Crear índice para la shard key propuesta en product_catalog

product_catalog.create_index([
    ("category", 1),
    ("seller_region", 1),
    ("_id", "hashed")
])

print("Índice para la shard key de product_catalog creado correctamente")

Índice para la shard key de product_catalog creado correctamente

```

Aplicar shard key a product_catalog:

Sharding teórico para `product_catalog`

El cluster utilizado en esta actividad corresponde a MongoDB Atlas Free Tier M0, por lo tanto no es posible ejecutar una configuración real de sharding sobre la base de datos.

En un ambiente productivo con MongoDB Sharded Cluster, primero se crearía el índice asociado a la shard key compuesta:

```
db.product_catalog.createIndex({
  "category": 1,
  "seller_region": 1,
  "_id": "hashed"
})
```

Sharding para `user_behavior`:

Sharding teórico para `user_behavior`

La colección `user_behavior` almacena eventos de navegación y comportamiento de usuarios. Esta colección puede crecer de forma masiva debido a eventos como visualización de productos, búsquedas, adición al carrito y navegación por categorías.

En un ambiente productivo con sharding habilitado, se crearía el siguiente índice:

```
db.user_behavior.createIndex({
  "userId": "hashed",
  "period": 1
})
```

```
[36]
✓ 2s
from random import randint, choice

user_behavior = db["user_behavior"]

user_behavior.delete_many({})

events = []

event_types = ["VIEW_PRODUCT", "ADD_TO_CART", "SEARCH", "VIEW_CATEGORY"]

for i in range(1, 501):
    document = {
        "_id": f"USER-{i:04d}-2026-06",
        "userId": randint(1, 200),
        "period": "2026-06",
        "events": [
            {
                "type": choice(event_types),
                "productId": f"PROD-{randint(1, 1000):04d}",
                "timestamp": datetime.now()
            }
        ]
    }
    events.append(document)

user_behavior.insert_many(events)

print("Total documentos user_behavior:", user_behavior.count_documents({}))
```

... Total documentos user_behavior: 500

```
[37]
✓ 0s
# Índice equivalente para user_behavior usando PyMongo

user_behavior.create_index([
    ("userId", "hashed"),
    ("period", 1)
])
```

Sharding para `product_reviews`:

Sharding teórico para `product_reviews`

La colección `product_reviews` almacena las reseñas completas de los productos. Esta colección puede crecer rápidamente, ya que un producto popular puede acumular miles de reseñas.

En un ambiente productivo con sharding habilitado, se crearía un índice hashed sobre `product_id`:

```
db.product_reviews.createIndex({
  "product_id": "hashed"
})
```

índice equivalente para product_reviews usando PyMongo

```
product_reviews.create_index([
  ("product_id", "hashed")
])

print("Índice teórico de shard key para product_reviews creado correctamente")
```

Índice teórico de shard key para product_reviews creado correctamente

19. Ventajas de la estrategia propuesta

La estrategia propuesta ofrece las siguientes ventajas:

1. Permite escalar horizontalmente las colecciones de mayor crecimiento.
2. Reduce la carga del nodo Primary mediante lecturas desde Secondary.
3. Mejora disponibilidad mediante replica sets en múltiples zonas.
4. Reduce riesgo de hotspots mediante shard keys compuestas y hashed.
5. Separa operaciones críticas de operaciones analíticas.
6. Mantiene consistencia fuerte en PostgreSQL para pagos, órdenes e inventario.
7. Permite consistencia eventual controlada en MongoDB.
8. Facilita el crecimiento futuro de catálogo, reseñas y comportamiento de usuario.
9. Se alinea con la arquitectura orientada a eventos de Ecommify.
10. Permite monitorear replication lag, consultas lentas y distribución de chunks.

20. Trade-offs de la estrategia propuesta

Decisión	Beneficio	Riesgo	Mitigación
Usar replica set de 3 nodos	Alta disponibilidad	Mayor costo operativo	Uso multi-AZ controlado
Leer catálogo desde Secondary	Reduce carga del Primary	Puede haber datos desactualizados	Usar Primary en operaciones críticas

Usar write concern majority	Mayor durabilidad	Mayor latencia	Aplicarlo solo en operaciones importantes
Usar w:1 para eventos	Mejor rendimiento	Posible pérdida parcial	Reprocesamiento asíncrono
Usar shard key compuesta	Balance entre consulta y distribución	Mayor complejidad	Índices complementarios
Usar _id hashed	Reduce hotspots	Menor eficiencia en consultas por rango	Combinar con categoría y región
Sharding en logs y comportamiento	Escalabilidad	Mayor complejidad operativa	Monitoreo y políticas de archivado

21. Conclusiones

La planificación de sharding y replica sets para Ecommify permite diseñar una arquitectura MongoDB preparada para alta disponibilidad, tolerancia a fallos y crecimiento horizontal.

El replica set propuesto de tres nodos en múltiples zonas de disponibilidad permite continuidad del servicio ante fallos del nodo Primary. Además, la separación de lecturas mediante read preferences ayuda a descargar el Primary y mejorar el rendimiento de consultas frecuentes del catálogo.

La estrategia de write concerns diferencia operaciones críticas de eventos analíticos. Para operaciones como creación de productos, actualización de precios y registro de reseñas se recomienda majority. Para eventos de navegación, búsquedas y recomendaciones se puede utilizar w:1, ya que son datos de alto volumen y pueden reprocesarse.

El análisis de distribución realizado sobre los 1000 productos insertados muestra una distribución equilibrada por categoría y región del vendedor. Sin embargo, en un escenario productivo real podrían aparecer concentraciones en categorías o regiones específicas. Por esta razón, se recomienda una shard key compuesta para product_catalog basada en category, seller_region y _id hashed.

La consistencia eventual es aceptable para recomendaciones, logs, navegación y métricas aproximadas, pero debe mitigarse en operaciones sensibles mediante lecturas desde Primary, write concern majority, sesiones con causal consistency y monitoreo del replication lag.

Finalmente, debido a las limitaciones del free tier de MongoDB Atlas, la estrategia de sharding y replica sets se documenta como una aproximación teórica. La validación práctica se apoya en los datos insertados, los pipelines ejecutados y el análisis de distribución realizado en la Etapa 1.

22. Referencias

- MongoDB, Inc. Data Modeling Patterns. MongoDB Manual.
- MongoDB, Inc. Sharding. MongoDB Manual.
- MongoDB, Inc. Replica Set Deployment Architectures. MongoDB Manual.
- MongoDB, Inc. Read Preference. MongoDB Manual.
- MongoDB, Inc. Write Concern. MongoDB Manual.
- Documento Técnico de Diseño – Ecommify.